Homework 07 Solutions

Prerith Arunkumar (prerith2)

Manual LDA on the South Africa heart data, QDA with marginal screening on the handwritten digits, a simulation comparing LDA and QDA, and an LDA/QDA analysis skill applied to the heart data.

Use of AI tools. I used Claude (Claude Code) to draft the `R` code and exposition for Questions 1 through 3 and to run them. For Question 4, I designed the skill with Claude, and a separate Claude agent applied it to the data. Every number below comes from the code output shown in this report, and I checked the numbers quoted in the text against it.

```
library(MASS)
```

Question 1: LDA

```
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
n <- nrow(X); cls <- sort(unique(y)); K <- length(cls)

pi_hat <- sapply(cls, function(k) mean(y == k))              # K priors
mu_hat <- sapply(cls, function(k) colMeans(X[y == k, ]))     # p x K class means
Sigma_hat <- Reduce(`+`, lapply(seq_len(K), function(j) {
  Z <- sweep(X[y == cls[j], ], 2, mu_hat[, j])               # center within class j
  crossprod(Z)
})) / (n - K)                                                # pooled, denominator n - K
names(pi_hat) <- colnames(mu_hat) <- paste0("chd=", cls)
round(pi_hat, 4)
```

```
 chd=0  chd=1
0.6537 0.3463
```

```
round(mu_hat, 3)
```

```
            chd=0   chd=1
sbp       135.460 143.738
tobacco     2.635   5.525
ldl         4.344   5.488
adiposity  23.969  28.120
famhist     1.318   1.600
typea      52.368  54.494
obesity    25.737  26.623
alcohol    15.931  19.145
age        38.854  50.294
```

```
round(Sigma_hat, 3)
```

```
              sbp tobacco    ldl adiposity famhist   typea obesity alcohol
sbp       405.435  14.585  4.581    49.164   0.337 -15.588  18.939  64.400
tobacco    14.585  19.243  0.763     7.538   0.016  -2.057   1.833  20.517
ldl         4.581   0.763  4.001     6.033   0.092   0.345   2.660  -2.533
adiposity  49.164   7.538  6.033    56.753   0.433  -5.310  22.708  16.119
famhist     0.337   0.016  0.092     0.433   0.226   0.081   0.184   0.768
typea     -15.588  -2.057  0.345    -5.310   0.081  95.566   2.640   7.960
obesity    18.939   1.833  2.660    22.708   0.184   2.640  17.615   4.689
alcohol    64.400  20.517 -2.533    16.119   0.768   7.960   4.689 598.277
age        95.134  22.765  6.479    60.509   0.997 -20.278  15.697  27.886
              age
sbp        95.134
tobacco    22.765
ldl         6.479
adiposity  60.509
famhist     0.997
typea     -20.278
obesity    15.697
alcohol    27.886
age       184.132
```

(a) The priors are $\hat{\pi}_0 = 0.6537$ and $\hat{\pi}_1 = 0.3463$ ($n = 462$). The class means are the columns of the 9×2 matrix above, and the pooled covariance is the 9×9 matrix $\hat{\Sigma} = \sum_k \sum_{i:y_i=k} (\mathbf{x}_i - \hat{\mu}_k)(\mathbf{x}_i - \hat{\mu}_k)^\mathsf{T}/(n-K)$.

```
Sigma_inv <- solve(Sigma_hat)                                  # 9 x 9
lin   <- X %*% Sigma_inv %*% mu_hat                            # n x K: x' Sigma^-1 mu_k
const <- -0.5 * diag(t(mu_hat) %*% Sigma_inv %*% mu_hat) + log(pi_hat)   # K
scores <- sweep(lin, 2, const, "+")                           # n x K scores
c(dim(X), dim(mu_hat), dim(Sigma_inv), dim(scores))
```

```
[1] 462   9   9   2   9   9 462   2
```

```
pred_manual <- cls[max.col(scores)]
cm_manual <- table(observed = y, predicted = pred_manual)
cm_manual
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

(b) The dimensions check out: $\mathbf{X}$ is 462×9, the means form a 9×2 matrix, $\hat{\Sigma}^{-1}$ is 9×9, and the score matrix is 462×2. In-sample, the manual LDA rule misclassifies 117 of 462 observations (training error 0.253).

```
fit_lda <- lda(X, grouping = y)              # priors default to the class proportions
pred_mass <- predict(fit_lda, X)$class
table(observed = y, predicted = pred_mass)
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

```r
all(as.character(pred_mass) == as.character(pred_manual))
```

```
[1] TRUE
```

(c) `MASS::lda` gives exactly the same confusion matrix, and its predicted label agrees with the manual rule for every one of the 462 observations. This is expected: `lda` uses the same priors, class means, and pooled covariance with denominator $n - K$, and it classifies to the largest posterior, which is the largest score.

Question 2: QDA and marginal screening

```r
ztrain <- read.csv("data/zip-train.csv.gz", header = FALSE)
ztest  <- read.csv("data/zip-test.csv.gz",  header = FALSE)
Xtr <- as.matrix(ztrain[, -1]); ytr <- factor(ztrain[, 1])
Xte <- as.matrix(ztest[, -1]);  yte <- factor(ztest[, 1])
pix_var <- apply(Xtr, 2, var)                       # marginal variance of each pixel (training data)
ord <- order(pix_var, decreasing = TRUE)

qda_screen <- function(m) {
  sel <- ord[1:m]
  fit <- qda(Xtr[, sel], grouping = ytr)
  pred <- predict(fit, Xte[, sel])$class
  list(cm = table(observed = yte, predicted = pred), err = mean(pred != yte))
}
q40 <- qda_screen(40)
q40$cm
```

```
        predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
       5  12   0   3   5   1 131   0   0   6   2
       6   0   0   8   0   1   8 148   0   5   0
       7   1   0   6   1   6   0   0 124   2   7
       8   1   0  10  12   3   2   0   0 137   1
       9   0   0   1   2   7   1   1   4   9 152
```

```r
q40$err
```

```
[1] 0.1554559
```

(a) With the 40 highest-variance pixels, QDA fits without error, and its test misclassification rate is 0.1555 on the 2007 test digits.

(b) Yes, the method works. With all 256 pixels, QDA needs a separate 256×256 covariance matrix for each digit. Many pixels near the border are -1 (blank) for nearly every image of a given digit, so their within-class variance is zero or close to it, and the class covariance matrices are singular. Screening on marginal variance keeps the pixels whose ink varies most across images. These pixels lie in the central columns of the image, and since the variation comes partly from differences between digits, they also tend to vary within each digit. With 40 pixels, each class covariance has $40 \cdot 41/2 = 820$ parameters, and the smallest class has 542 training images, so each covariance can be

estimated and inverted. The digit-1 covariance is still close to singular, because one of the selected pixels is almost constant among the 1s, but `qda` fits it. The error of about 16% is still much higher than what a good digit classifier achieves. I think this is partly because QDA estimates 10×820 covariance parameters, which adds variance, and partly because the other 216 pixels are dropped.

```
q10 <- qda_screen(10)
q10$cm
```

```
          predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
       2  19   2  95  14  14   3  22  21   5   3
       3   5   0   6 103   3  42   0   0   7   0
       4   1   5  14   0  95   2   4  15  17  47
       5  15   0   6  25   4  96   4   2   8   0
       6  41   0  15   4   0  18  78   7   5   2
       7   4   1   6   0   6   0   1 122   0   7
       8   1   2  11  36  12  11   4   1  74  14
       9   0   2   9   2  20   0   0  16  10 118
```

```
q10$err
```

```
[1] 0.3423019
```

(c) With only 10 pixels, the test error rises to 0.3423, about twice the 40-pixel error. The covariance estimates are now very stable, but 10 pixels do not carry enough information to tell the digits apart. For example, 6 is often predicted as 0 and 3 as 5. The bias from discarding informative pixels outweighs the reduction in variance, so the 40-pixel model is better.

Question 3: Compare LDA and QDA

```
# n observations from each class. In class 1, (X1, X2) has mean (-1, 0) and
# correlation rho; in class 2, mean (1, 0) and correlation -rho.
# X3, ..., X10 are independent N(0, 1) in both classes.
gen <- function(n, rho) {
  y <- rep(1:2, each = n)
  X <- matrix(rnorm(2 * n * 10), 2 * n)
  r <- ifelse(y == 1, rho, -rho)
  X[, 2] <- r * X[, 1] + sqrt(1 - rho^2) * X[, 2]   # corr(X1, X2) = r, both unit variance
  X[, 1] <- X[, 1] + ifelse(y == 1, -1, 1)
  list(X = X, y = factor(y))
}
test_err <- function(train, test) {
  pr <- c(0.5, 0.5)
  c(LDA = mean(predict(lda(train$X, train$y, prior = pr), test$X)$class != test$y),
    QDA = mean(predict(qda(train$X, train$y, prior = pr), test$X)$class != test$y))
}
rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)
set.seed(432)
```

```r
one_run <- sapply(rhos, function(rho) test_err(gen(100, rho), gen(1000, rho)))
colnames(one_run) <- paste0("rho=", rhos)
round(one_run, 4)
```

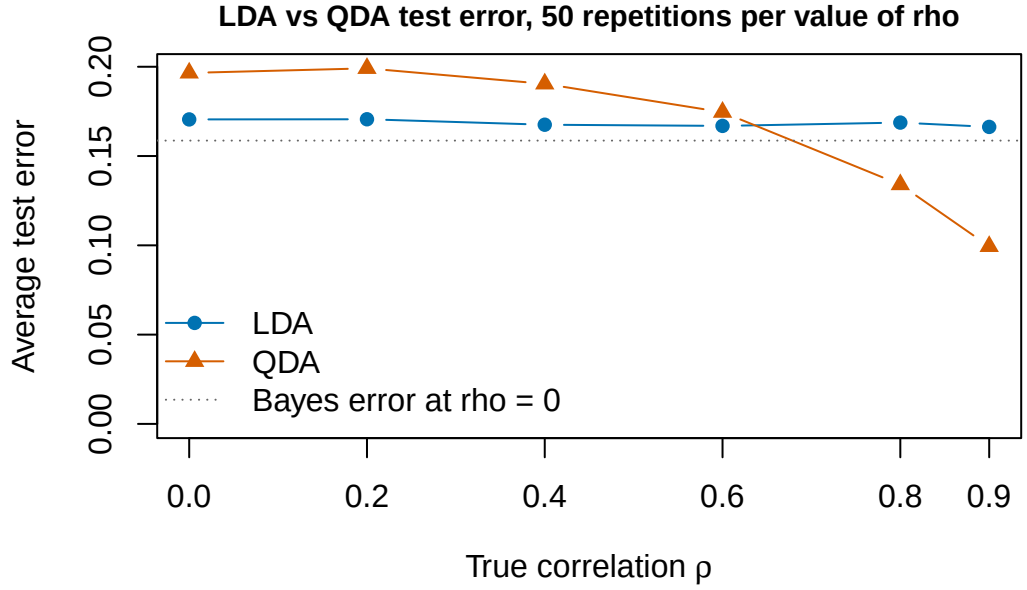
```
    rho=0 rho=0.2 rho=0.4 rho=0.6 rho=0.8 rho=0.9
LDA 0.157  0.1685  0.1750  0.1705  0.1855  0.1800
QDA 0.176  0.1990  0.1995  0.1675  0.1495  0.0955
```

(a) The seed is 432. In each setting, LDA and QDA are trained on the same 200 observations and tested on the same 2,000.

```r
avg <- sapply(rhos, function(rho)
  rowMeans(replicate(50, test_err(gen(100, rho), gen(1000, rho)))))
colnames(avg) <- paste0("rho=", rhos)
round(avg, 4)
```

```
     rho=0 rho=0.2 rho=0.4 rho=0.6 rho=0.8 rho=0.9
LDA 0.1705  0.1706  0.1675  0.1669  0.1687  0.1663
QDA 0.1966  0.1991  0.1904  0.1746  0.1340  0.0994
```

```r
par(mar = c(4, 4, 2, 1))
matplot(rhos, t(avg), type = "b", pch = c(16, 17), lty = 1, col = c("#0072B2", "#D55E00"),
        xaxt = "n", ylim = c(0, max(avg)), xlab = expression("True correlation " * rho),
        ylab = "Average test error",
        main = "LDA vs QDA test error, 50 repetitions per value of rho", cex.main = 0.95)
axis(1, at = rhos)
abline(h = pnorm(-1), lty = 3, col = "grey40")
legend("bottomleft", c("LDA", "QDA", "Bayes error at rho = 0"),
       col = c("#0072B2", "#D55E00", "grey40"), pch = c(16, 17, NA), lty = c(1, 1, 3),
       bty = "n")
```



(b) The figure shows the average test error over 50 repetitions for each ρ. The dotted line at $\Phi(-1) = 0.159$ is the Bayes error when $\rho = 0$, where the true boundary is $x_1 = 0$.

(c) When ρ is small, LDA is better (0.171 vs 0.197 at $\rho = 0$). The two class covariances are equal or nearly equal, so the shared-covariance assumption is correct or almost correct, and LDA has little bias. QDA estimates two full 10×10 covariance matrices (110 parameters) from 100 observations per class instead of one pooled matrix (55 parameters) from 200, so it pays a large variance cost for flexibility it does not need. LDA's error is nearly flat in ρ: the pooled covariance averages ρ and $-\rho$ to about 0, so LDA always fits a boundary close to $x_1 = 0$ and cannot use the correlation. As ρ grows, the classes differ more in their covariance. Because the two covariance matrices have the same determinant, the difference of the class-1 and class-2 quadratic forms is $4x_1(1 - \rho x_2)/(1 - \rho^2)$, so the Bayes rule assigns class 1 exactly when $x_1(1 - \rho x_2) < 0$. The Bayes boundary is the pair of lines $x_1 = 0$ and $x_2 = 1/\rho$: above $x_2 = 1/\rho$ the correct side of $x_1 = 0$ flips. For small ρ that second line is far out in the tails and hardly matters, but at $\rho = 0.9$ it is at $x_2 \approx 1.11$, where many observations lie, and a single linear boundary cannot follow it. LDA's bias then dominates, and QDA's reduction in bias outweighs its higher variance. The curves cross between $\rho = 0.6$ and 0.8, and at $\rho = 0.9$ QDA's error is 0.099 against 0.166 for LDA.

The eight noise covariates carry no class information, but both methods still estimate their means and their covariances with everything else. Each fitted mean difference and covariance entry for a noise variable is estimation error that slightly shifts and bends the fitted boundary, so it raises the variance of the classifier. This matters most for QDA. Its covariance parameters grow with p^2 in each class: the noise variables add $2(55 - 3) = 104$ of QDA's 110 covariance parameters. This is why QDA loses to LDA at small ρ by more than the two-variable problem alone would suggest.

Question 4: Create and use an LDA/QDA analysis skill

The skill definition and how I invoked it come first; the one-page analysis report it produced is on the last page.

Skill definition and invocation

File size and explicit invocation. The skill is a single file, `.claude/skills/lda-qda-report/SKILL.md`, of 4727 bytes (limit 5,000). It is a Claude Code project skill. The front matter sets `disable-model-invocation: true`, so Claude never loads it on its own; it runs only when I call it by name. Its description also states that it is for explicit invocation only. I applied it by giving a Claude agent the following prompt, quoted in full (line breaks added to fit the page):

```
I am explicitly invoking my skill `lda-qda-report`. Read
/Users/preritharunkumar/stat432-fall2026/.claude/skills/lda-qda-report/SKILL.md in full and follow
it exactly.

Inputs:
- Data: /Users/preritharunkumar/stat432-fall2026/homework/week-07/data/saheart.csv (relative path
  from the report: data/saheart.csv)
- Response: `chd` (two classes, 0/1)
- Predictors: all nine other columns as numeric; code `famhist` as 1 = "Absent", 2 = "Present" (e.g.
  `heart$famhist <- factor(heart$famhist, levels = c("Absent","Present")); X <- data.matrix(heart[,
  names(heart) != "chd"])`).
- Language: R (MASS is installed). Seed: 432.
- Goal: the best predictive performance within the LDA/QDA methods the skill allows; ordinary LDA
  and QDA must be included as comparisons.
- Output: write the report section as a Quarto fragment to
  /Users/preritharunkumar/stat432-fall2026/homework/week-07/q4-report.qmd. It will be included in a
  PDF report (10pt, 0.8in margins, letter paper) under a heading "### Analysis report (one page)",
  so do not write a YAML header or top-level heading; start with the setup text. Use chunk labels
  prefixed `q4-`. Base R graphics only (no ggplot). Use `knitr::kable` for tables.

To check the one-page requirement, make a scratch wrapper .qmd in /private/tmp/claude-501/-Users-pre
ritharunkumar-stat432-fall2026/65ca89a9-fa9e-4c93-aeed-5db4cd463c1d/scratchpad/q4check/ with the PDF
format settings above (documentclass article, geometry margin=0.8in, fontsize 10pt, echo: true,
code-overflow: wrap) that includes your fragment with `{{< include ... >}}` (copy the data folder
next to it), render with `quarto render`, and confirm the section fits on one page (use
pdfinfo/pdftotext). Do not modify the SKILL.md. Do not edit any other files in the repo.

Return: seed, selected model, its CV error and SE, test error, a short list of the CV table, page
count of your check render, and any issues.
```

Checking the analysis and revisions. I read the agent's code and checked that the test set is drawn before the CV folds and is used only once, that all 21 candidates use the same stratified folds, that standardization and PCA are estimated inside each training fold, and that the blend averages posteriors. A second, independent agent reimplemented the analysis and reproduced every CV error, the selected model, and the test error. I did not change the candidates, the folds, or the selection rule. The only edits after the run used no test-set results: I reflowed the line breaks in `SKILL.md` so it prints within the margins (no change in wording), and I corrected two sentences in the agent's report that overstated what the CV results show (the PCA gain over LDA is within about one SE, and the non-normality remark was not backed by any output).

Complete `SKILL.md` (the front-matter description is one line in the file and is wrapped here for display).

```
---
name: lda-qda-report
description: Run a cross-validated LDA/QDA classification analysis on a user-supplied dataset
  and write a one-page statistical report. Use only when the user explicitly invokes
  lda-qda-report and names the data, response, and predictors.
disable-model-invocation: true
---

# lda-qda-report

Analyze one classification dataset with discriminant-analysis methods and produce a
one-page report. The user supplies the data file, the response, the predictors (with any
coding), the language (default R), the seed, and where to write the output. If any of
these is missing, ask; never guess variable roles.

## Allowed models

Every classifier must be LDA, QDA, or a combination of them. Fit the candidates below with
the same priors (empirical class proportions of the training data in use) and classify to
the largest posterior probability.

1. **LDA** on all predictors (R `MASS::lda`; Python
   `LinearDiscriminantAnalysis(solver="svd")`).
2. **QDA** on all predictors (R `MASS::qda`; Python `QuadraticDiscriminantAnalysis`).
3. **Blend**: posterior `w * P_LDA + (1 - w) * P_QDA` for `w` in {0.25, 0.5, 0.75} (0 and
   1 are QDA and LDA).
4. **PCA + LDA** and **PCA + QDA**: standardize the predictors, project on the first `d`
   principal components, then fit LDA or QDA, for `d` from 1 to the number of predictors
   minus one.

Do not add other model families (logistic regression, KNN, trees, ...), and do not tune
the classification threshold.

## Procedure

1. Set and report one random seed at the top of the code.
2. Read the data and apply exactly the coding the user gave. Report n, p, and the class
   counts.
3. **Reserve a test set first**: a stratified random 25% of observations. Do not look at,
   summarize, or use it again until step 7.
4. Create one set of stratified 10-fold CV folds on the training set. Use these same folds
   for every candidate and every tuning value.
5. Inside each fold, estimate everything from the fold's training part only:
   standardization means and SDs, PCA loadings, priors, means, covariances. Apply them
   unchanged to the held-out fold.
6. Record each candidate's CV misclassification rate (mean over folds) and its standard
   error (fold SD / sqrt(10)). Select the candidate with the smallest CV error; if a
   simpler candidate is within 0.005 of it, prefer the simpler one (LDA < blend < QDA;
   fewer PCs first) and say so.
7. Refit the selected model and its preprocessing on the whole training set and evaluate
   it **once** on the test set. Report the test misclassification rate, the test confusion
   matrix (observed in rows, predicted in columns), and the majority-class error on the
   test set as a baseline.
8. If a covariance matrix is singular or a fit fails in a fold, report it and drop that
   candidate rather than silently regularizing.

Never change the candidate set, the folds, or the selection rule after seeing the test
result.
```

```
## Report (at most one printed page)

Write it as a Quarto section that renders inside the user's PDF report: the code chunk(s)
followed by text whose numbers are inline code results, not typed by hand. Keep code
compact and hide nothing the grader needs. Contents, in this order:

1. **Setup** (2-3 sentences): data, response, predictors, seed, split sizes, CV folds,
   candidates.
2. **Figure**: CV misclassification rate (y, starting at a sensible range) for each
   candidate family against its tuning value (number of PCs; blend weight on a separate
   small panel or as points), with +/- 1 SE bars and horizontal reference lines for plain
   LDA and QDA. One panel row, height <= 2.6 in, readable labels, sentence- case title,
   legend that hides no data, colorblind-safe colors.
3. **Table**: one row per candidate family at its best tuning value: family, setting, CV
   error, SE. Mark the selected row. Round errors to 3 decimals.
4. **Fitting results**: the selected model's test error, test confusion matrix, and
   baseline error; for an LDA based choice also list the 3 predictors with the largest
   absolute standardized discriminant coefficients.
5. **Findings** (3-4 sentences): which assumption (shared vs separate covariances,
   dimension reduction) the CV results support and why, and how test error compares with
   CV error.
6. **Limitations** (2-3 sentences): e.g. one split, CV variance, normality, class
   imbalance, the 0.5 rule.

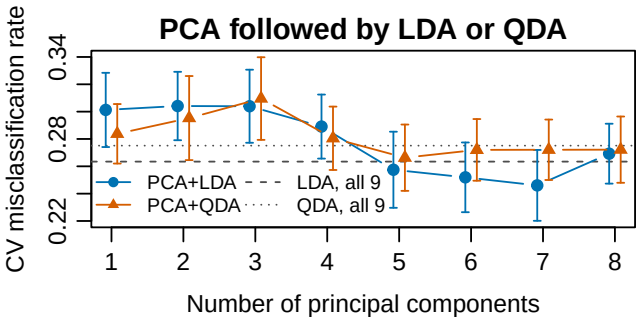
## Checks before finishing

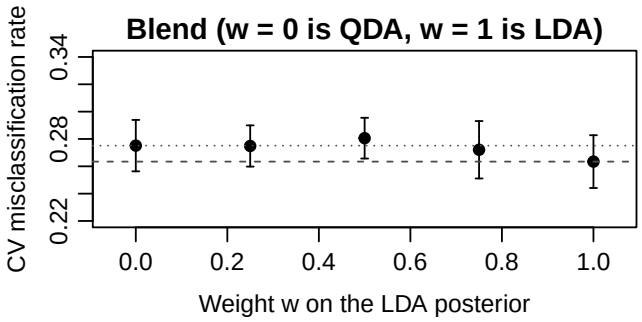
- Render the document and confirm the report section fits on one page; shorten text first,
  then the figure.
- Confirm every number in the text matches the code output.
- Confirm the test set was used only in step 7.
- Return the seed, the selected model, CV and test errors, and any issues you hit.
```

Analysis report (one page)

Setup. We predict `chd` (0/1) from the other nine variables as numeric (`famhist`: 1 = Absent, 2 = Present); seed 432. After reserving a stratified 25% test set, one set of stratified 10-fold CV folds on the training set compares LDA, QDA, blends $wP_{LDA} + (1-w)P_{QDA}$ (w = 0.25, 0.5, 0.75), and PCA of the standardized predictors (1 to 8 components) followed by LDA or QDA. Scaling, PCA, empirical priors, means and covariances use each fold's training part only; we predict the largest-posterior class.

```
library(MASS); set.seed(432); heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"]); y <- factor(heart$chd); n <- nrow(X); p <- ncol(X)
test <- unlist(lapply(split(1:n, y), function(i) sample(i, round(0.25 * length(i))))  # stratified 25% test set
Xtr <- X[-test, ]; ytr <- y[-test]; fold <- integer(length(ytr))                      # stratified 10 CV folds
for (k in levels(y)) { i <- which(ytr == k); fold[i] <- sample(rep(1:10, length.out = length(i))) }
post <- function(m, Xa, ya, Xb, d = NA) {   # fit on (Xa, ya) only, return posteriors for Xb
  if (!is.na(d)) { mu <- colMeans(Xa); s <- apply(Xa, 2, sd); pc <- prcomp(scale(Xa, mu, s))
    Xb <- scale(Xb, mu, s) %*% pc$rotation[, 1:d, drop = FALSE]; Xa <- pc$x[, 1:d, drop = FALSE] }
  predict(if (m == "lda") lda(Xa, ya) else qda(Xa, ya), Xb)$posterior }
cand <- rbind(data.frame(fam = "PCA+LDA", m = "lda", d = 1:(p - 1), w = NA),   # ordered simplest first
  data.frame(fam = "LDA", m = "lda", d = NA, w = 1), data.frame(fam = "Blend", m = NA, d = NA, w = c(.75, .5, .25)),
  data.frame(fam = "PCA+QDA", m = "qda", d = 1:(p - 1), w = NA), data.frame(fam = "QDA", m = "qda", d = NA, w = 0))
pr <- function(j, Xa, ya, Xb) with(cand[j, ], if (!is.na(d)) post(m, Xa, ya, Xb, d) else
  w * post("lda", Xa, ya, Xb) + (1 - w) * post("qda", Xa, ya, Xb))
cls <- function(P) max.col(P, ties.method = "first")   # class index with the largest posterior
err <- sapply(1:10, function(f) sapply(1:nrow(cand), function(j)
  mean(cls(pr(j, Xtr[fold != f, ], ytr[fold != f], Xtr[fold == f, ])) != as.integer(ytr[fold == f]))))
cand$cv <- rowMeans(err); cand$se <- apply(err, 1, sd) / sqrt(10)
sel <- which(cand$cv <= min(cand$cv) + 0.005)[1]   # simplest candidate within 0.005 of the best; refit it on all training data:
yhat <- factor(cls(pr(sel, Xtr, ytr, X[test, ])), 1:2, levels(y)); cm <- table(y[test], yhat)
test_err <- mean(yhat != y[test]); base_err <- mean(y[test] != names(which.max(table(ytr))))
if (cand$m[sel] %in% "lda") { d <- cand$d[sel]; Z <- scale(Xtr); pc <- prcomp(Z)   # standardized coefficients
  B <- if (is.na(d)) lda(Z, ytr)$scaling else pc$rotation[, 1:d] %*% lda(pc$x[, 1:d, drop = FALSE], ytr)$scaling
  top3 <- names(sort(abs(B[, 1]), decreasing = TRUE))[1:3] }
```





Family	Best setting	CV error	SE	Selected
LDA	all 9 predictors	0.263	0.019	
QDA	all 9 predictors	0.275	0.019	
Blend	w = 0.75	0.272	0.021	
PCA+LDA	7 PCs	0.246	0.026	**yes**
PCA+QDA	5 PCs	0.266	0.024	

Fitting results. n = 462, p = 9, classes 302 (0) / 160 (1); 346 training, 116 test. The selected model is **PCA+LDA with 7 PCs** (CV error 0.246, SE 0.026); no simpler candidate was within 0.005 of it. Refitted on all training data, its **test misclassification rate is 0.293**, versus 0.345 for always predicting the majority class. Test confusion matrix (rows observed 0/1, columns predicted 0/1): [64, 12; 22, 18]. Largest |standardized coefficients|: `famhist`, `tobacco`, `ldl`.

Findings. CV supports a shared covariance: LDA (0.263) beat QDA (0.275), blending in QDA did not help, and no PCA+QDA model beat LDA: separate covariances add variance, not accuracy. Keeping 5 to 7 of 9 components lowered LDA's CV error slightly, by less than about one SE (about 0.02). Test error is above CV error by 0.047, within the noise of CV and a small test set.

Limitations. One split with 116 test cases makes the test error noisy, and taking the minimum of 21 CV errors biases it downward. `famhist` is binary and the normal model is only an approximation for the other predictors, and the 0.5 posterior rule with 35% cases misses many of the cases.